

LUNCH AND LEARN • FIVE PRACTICES • 60 MINUTES

# Agentic engineering

Coding agents are fast. These five practices make them reliable.

THE ART OF THE POSSIBLE

# Agents can now take a ticket all the way to review.

STEP 01

## Plan

Reads the ticket and plans the change.

STEP 02

## Build

Writes code and tests, opens a pull request.

STEP 03

## Fix

Fixes whatever breaks the build.

STEP 04

## Report

Notes how sure it is, and why.

STEP 05

## Hand off

Waits for approval, leaves notes.

SO WHAT

The bottleneck has moved from writing the code to trusting the result.

THE ONE RULE • USE AGENTS ANY WAY YOU LIKE

# Reality must match the claim

YOUR ROLE • FLOOR AND CEILING

# AI raises your floor, but only you can raise your ceiling.

THE FLOOR

Anyone can now produce working-looking code in minutes. That part got cheap, for everyone.



THE CEILING

Knowing what to build, spotting what is wrong, deciding when it is good enough. That part is still yours.

SO WHAT

Everyone has a camera and a crew now. Not everyone makes a film.

YOUR ROLE • THE DIRECTOR

# Direct the work: watch the take, name the problem, give one note.

STEP 01

**Watch the take**

Read the actual change and run it before you react.



STEP 02

**Name the problem**

Say exactly what is off: "the retry hides the real error."



STEP 03

**Give one note**

Say it once, clearly, then let the agent work.

KEY INSIGHT

A precise note gets a fix. A vague one gets a guess.

YOUR ROLE • THE BAR

# Set the bar before the first take, and don't settle for the first pass.

SETTLE

"It compiles and the tests pass." The agent stops at the first green result, and so do you.



SET THE BAR

Decide what done means before work starts: it runs where users run it, someone looked at it, the docs match.

SO WHAT

Passing is the floor. You set the bar.

YOUR ROLE · MENTOR, DON'T MICROMANAGE

# Lead, follow, or get out of the way.

## Lead

When the goal is unclear, set the direction and say why.

## Follow

When the agent knows the ground better, let it propose a plan.

## Get out of the way

When limits and checks are in place, let it run.

### RECOMMENDATION

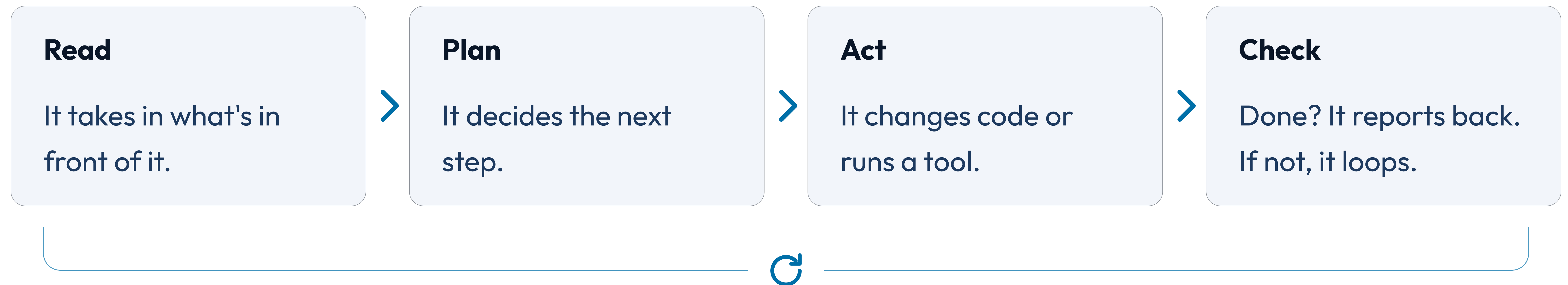
Don't try to control the agent. Mentor it, and put controls around it.

# Everything on a film set has a counterpart in agentic work.

| ON SET                    | WITH AGENTS                            | PRACTICE      |
|---------------------------|----------------------------------------|---------------|
| Script and shot list      | The plan and the instruction file      | Context       |
| One clear note per take   | One precise request                    | Autonomy      |
| The dailies               | Proof from the real thing              | Verification  |
| Continuity                | Checks that catch junk before it ships | Verification  |
| Production notes          | Decision log and follow-up files       | Learning      |
| Casting and the final cut | An agent roster; you approve the merge | Orchestration |

## HOW AN AGENT WORKS

# A coding agent loops until it thinks it is done.



### TAKEAWAY

Every practice today makes "thinks it is done" match "is done."

# Five practices, one for each question you will face.

---

01 Context: control what the agent sees

---

02 Autonomy: decide what it may do alone

---

03 Verification: make every claim prove itself

---

04 Learning: turn mistakes into rules

---

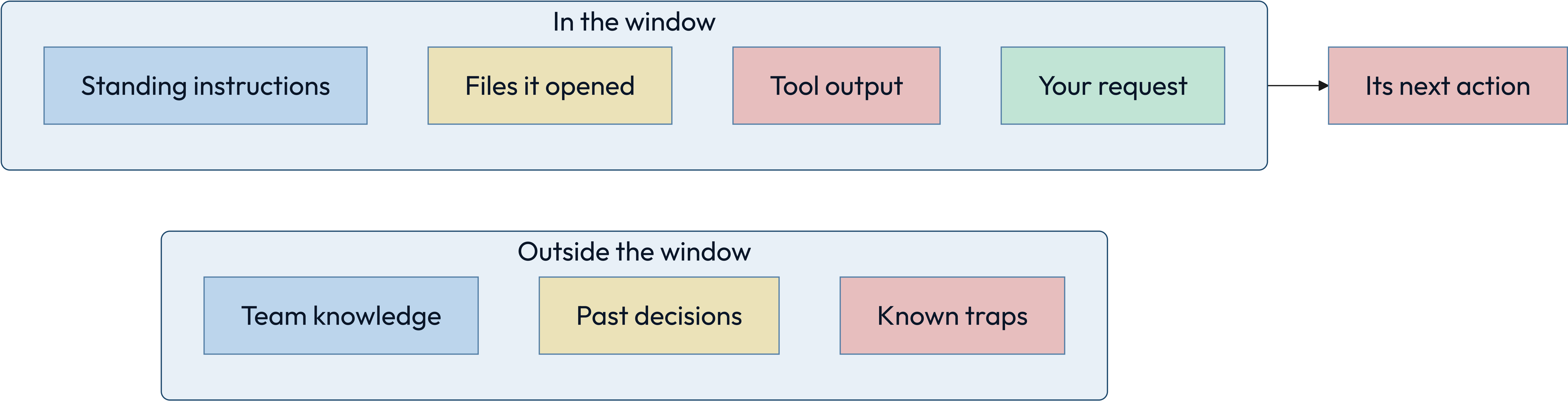
05 Orchestration: use many agents without losing control

---

PRACTICE 01 · THE SCRIPT AND THE SHOT LIST

# Context engineering

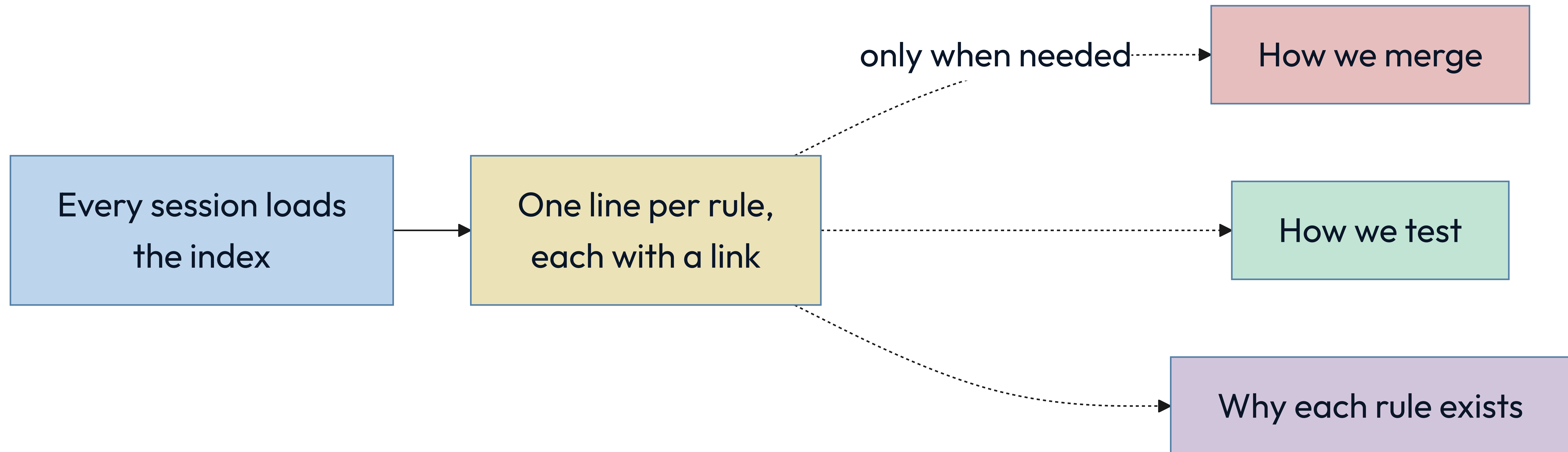
# An agent sees only its context window, the text in front of it.



## IMPLICATION

What you put in this window decides most of the agent's quality.

## Keep the always-loaded file short, and link to the detail.



### KEY INSIGHT

Load the index every time, and open the detail only when the task needs it.

# Route the agent to the right document before it starts work.

*## Read these before working in an area*

|                              |                             |  |
|------------------------------|-----------------------------|--|
| Working on...                | Read first                  |  |
| -----                        | -----                       |  |
| Branching, merging, releases | engineering/workflow.md     |  |
| Tests, hooks, CI             | engineering/development.md  |  |
| Something behaving strangely | engineering/gotchas.md      |  |
| Building a new script        | engineering/capabilities.md |  |
| A past decision              | engineering/decisions/      |  |

# Four habits keep the window full of what matters.

---

**Read by section:** list the headings, then open only what you need.

---

**Delegate big reads:** a helper agent reads the log and returns a summary.

---

**Quiet the tools:** print failures in full and successes as a dot.

---

**Measure first:** check what real sessions load before you optimize.

## CONTEXT • A QUIETER TEST REPORT

99.8%

less for the agent to read per test run:  
657,806 tokens down to 1,182.

# Claude Code gives you a setting for each of these habits.

---

Standing instructions: `CLAUDE.md`, linking detail with `@docs/file.md`.

---

Guidance on demand: a skill in `.claude/skills/<name>/SKILL.md`.

---

Big reads elsewhere: a helper agent (subagent) in `.claude/agents/`.

---

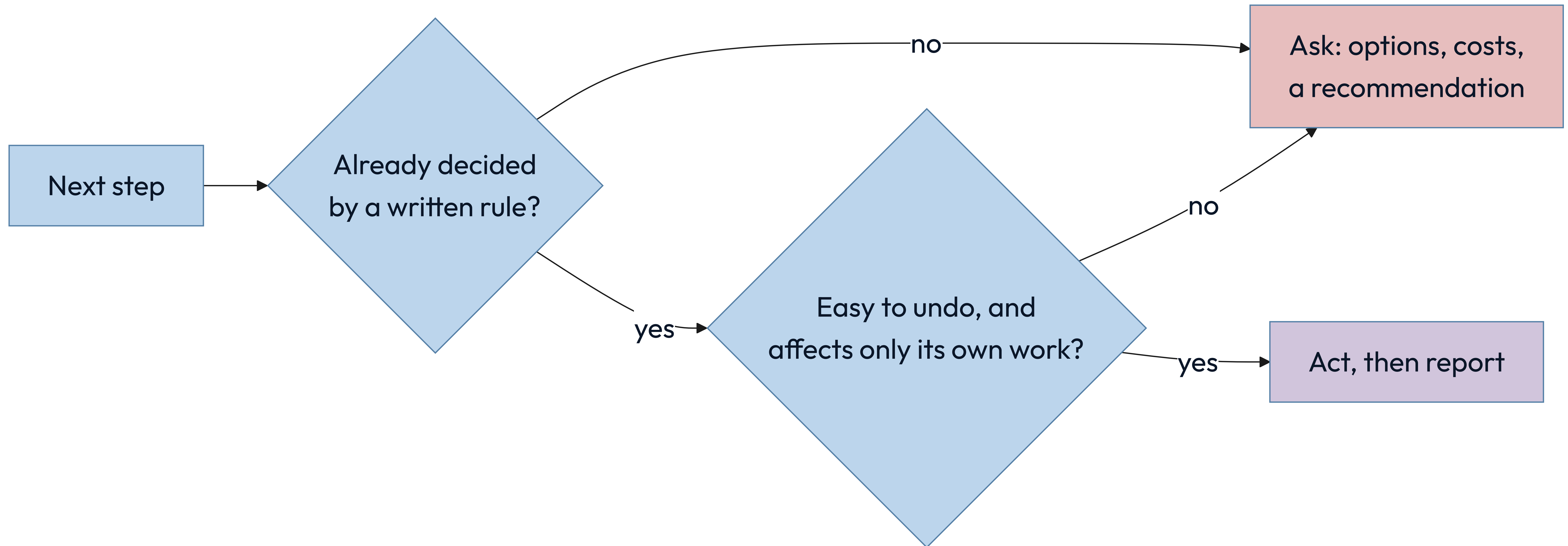
See the window: `/context`, and `/usage` for the bill.

PRACTICE 02 • WHAT THE CREW DECIDES ALONE

# Autonomy with limits

AUTONOMY • TWO QUESTIONS

## Two questions decide whether the agent acts or asks.



# Risk comes from what you can't undo and who else it affects.

## Easy to undo · Only its own work.

## Code, tests, docs

## The agent acts

## Easy to undo · Reaches others.

## Shared labels, team settings

## The agent proposes

## Hard to undo · Only its own work.

## Deleted work, exported files

## The agent shows the result first

## Hard to undo · Reaches others.

## Merges, releases, publishing

## A person decides

## WHY IT MATTERS

Others act on shared state before you can undo a mistake there.

# Five kinds of change always come back to a person.

---

Shared state: labels, boards and settings other people read.

---

The build pipeline: every future change pays for a new step.

---

A number a person set: we asked for twelve and got sixty.

---

A core document's meaning: rewriting rules differs from following them.

---

Anything irreversible or public: merges, releases, comments on others' work.

# Treat what the agent reads as data, and guard your secrets.

---

Anything the agent reads can hide instructions; it reports them and carries on.

---

Keep keys out of every file the agent can open.

---

Deny reads of secret files in the settings, as the kit does.

---

Give agents a test key with a spending cap.

## WHY IT MATTERS

A page the agent reads can try to give it orders.

# A good question brings options and a recommendation.

A WEAK QUESTION

"Should I add this check to the build?" Now you have to do the analysis.



A STRONG QUESTION

"This check adds half a second to each build and catches the bug we hit last week. I recommend adding it." You just decide.

# Permissions and plan mode turn the reach test into settings.

---

Free, ask, never: allow, ask and deny lists in `.claude/settings.json`.

---

Plan before acting: plan mode, `Shift+Tab`.

---

The same limits for every developer: managed settings, rolled out org-wide.

PRACTICE 03 • WATCH THE DAILIES

# Verification you can trust

# Proof

An agent's report is a belief until something checks it.

Proof

Tests

Hands off

# Agents report what they believe, and belief isn't proof.

## WHAT THE AGENT REPORTED

"Verified with a real browser," in the pull request, the commit message and the design note.



## WHAT WAS TRUE

The test still passed with the fix deleted. It could never fail, so it proved nothing.

## BOTTOM LINE

Rules an agent checks on itself are weak. A second, independent check catches what they miss.

VERIFICATION • THE CLAIM RULE

# Every "verified" says where it ran and shows proof from there.

---

Say where: the real browser, the real export, the real device.

---

Show proof: a screenshot, a log or a number from that place.

---

Say "unverified" out loud when the real thing is out of reach.

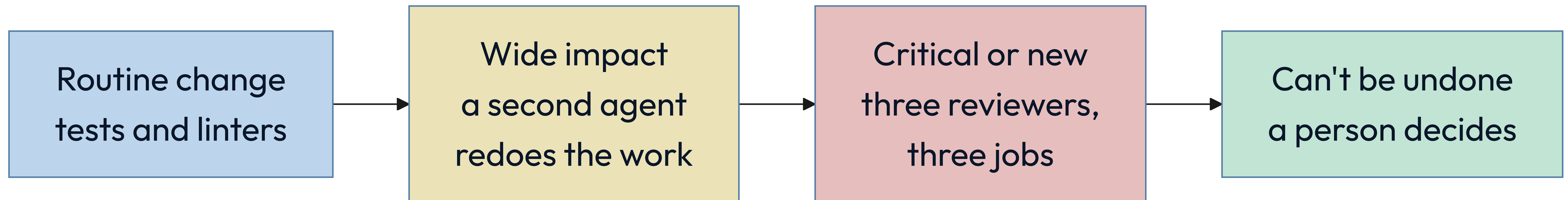
---

Treat a passing build as partial: it proves only what the build runs.

VERIFICATION • THE LADDER

# Match the checking to the damage a mistake could do.

---



# Tests

A test earns trust only when it can fail.

Proof

**Tests**

Hands off

# Beyond unit tests, each kind of test answers a different question.

**Mutation test: if I break the code on purpose, does a test fail?**

**Metamorphic test: if I change the input in a known way, does the output follow?**

**Visual diff: does it look the same as before, unless I meant it to change?**

**Benchmark: did it get slower, and by how much?**

**Fuzz test: does strange or random input break it?**

## WHY IT MATTERS

A check that always passes looks exactly like a check that works.

## VERIFICATION • TAUTOLOGICAL TESTS

# A test that repeats the code's own math can't catch its mistakes.

## REPEATS THE CODE

```
test('adds sales tax', () => {  
  // Same formula as the code under test.  
  // If the rate is wrong, both are wrong,  
  // and this still passes.  
  expect(tax(100)).toBe(100 * TAX_RATE);  
});
```

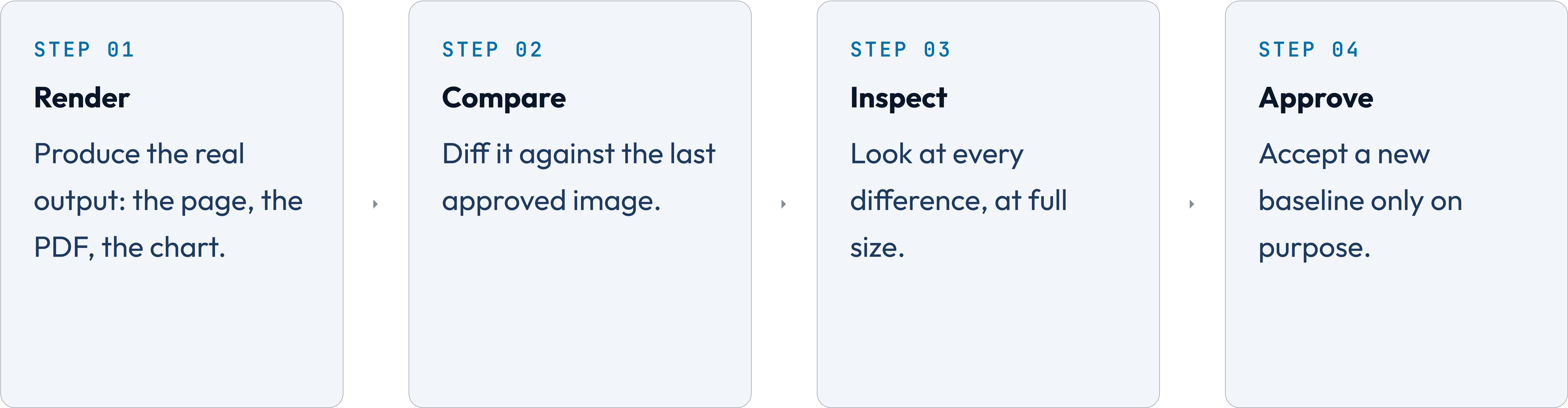
## CHECKS A KNOWN ANSWER

```
test('adds sales tax', () => {  
  // Worked out by hand from the tax table.  
  // If the code is wrong, this fails.  
  expect(tax(100)).toBe(8.25);  
});
```

## WHY IT MATTERS

Agents write tests that pass. Your job is to make sure they can fail.

# Nothing proves it looks right like rendering it and looking.



**OUR VIEW**

Tests check what you thought to check. A rendered picture shows everything.

# Hands off

Checks that run themselves let you leave the room.

Proof

Tests

Hands off

# Hooks and helper agents run the checks without being asked.

---

Checks after every edit: a `PostToolUse` hook.

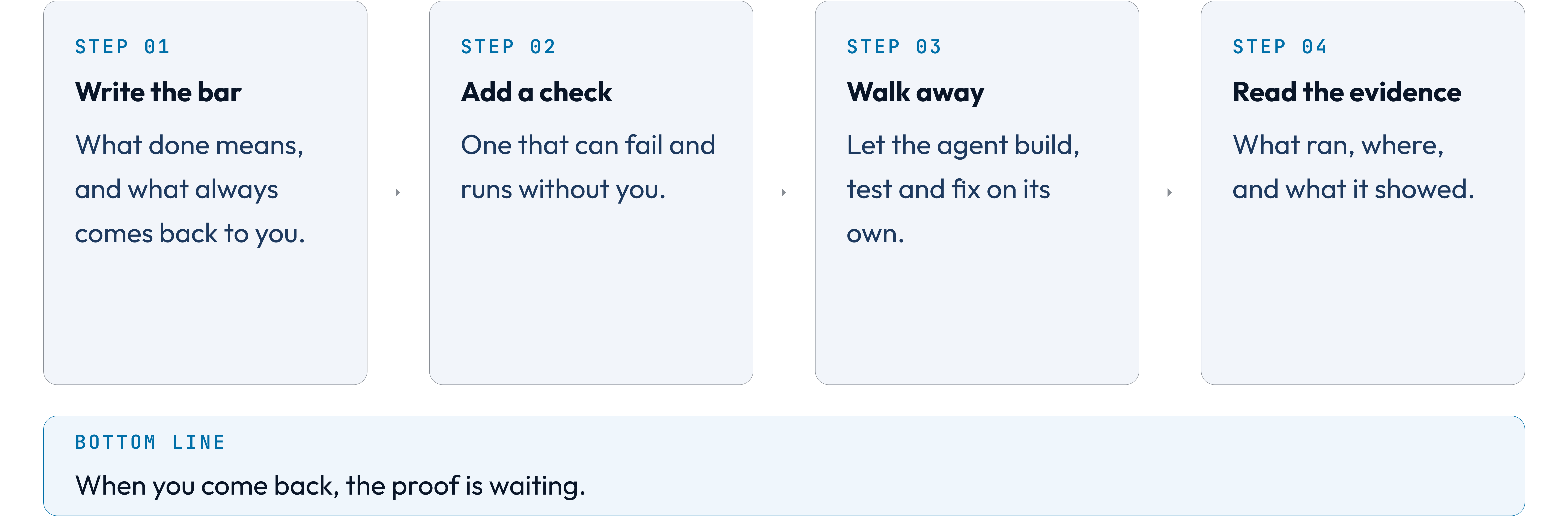
---

No "done" until the tests pass: a `Stop` hook that exits with code 2.

---

An independent reviewer: a read-only helper agent, or `/code-review`.

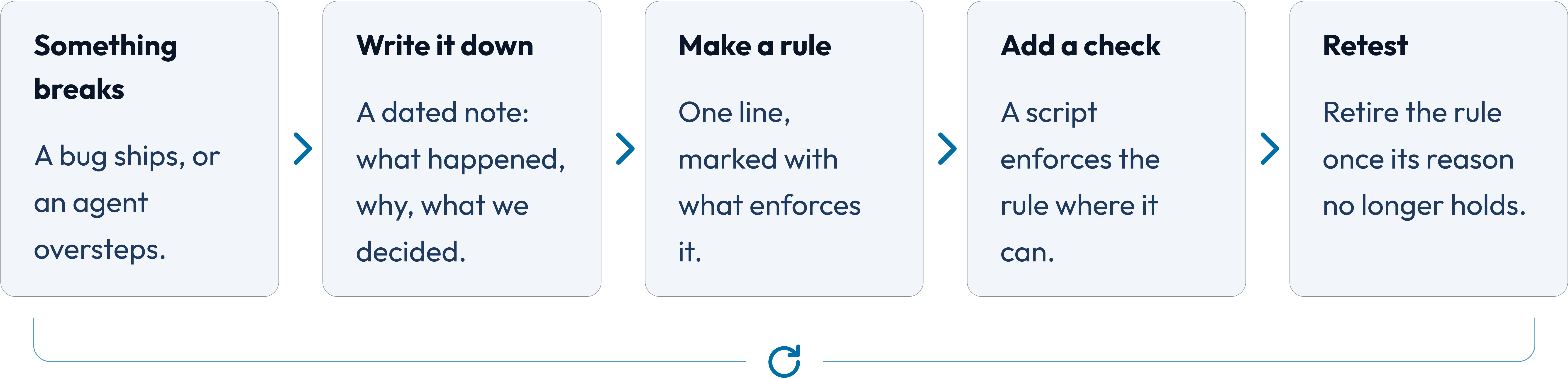
# With a bar and checks in place, you can leave the room.



PRACTICE 04 • KEEP CONTINUITY

# A system that learns

# Every rule should travel one loop, from mistake to retirement.



KEY INSIGHT

Copy the loop, and let your own mistakes write your rules.

# A decision note records why, so nobody has to guess later.

```
---  
status: shipped  
summary: Allow real database calls in tests; the ban had no evidence  
---
```

```
# Retire the rule against database calls in tests
```

|          |                                                    |
|----------|----------------------------------------------------|
| Symptom  | The rule forced slow, fragile mocks for two months |
| Claim    | Real database calls make tests flaky               |
| Evidence | 500 runs against a test database: 0 flaky          |
| Decision | Retire the rule; keep its number unused            |

# Three kinds of document answer three different questions.

---

**Proposal: which option should we pick? It expires once someone decides.**

---

**Decision record: why is it this way? A newer record replaces it.**

---

**Spec: what must every implementation do? You edit it so it stays true.**

# Before each merge, the agent grades itself by its weakest area.

Pre-merge: add retry to failed exports

|            |                                        |
|------------|----------------------------------------|
| WHAT       | One automatic retry on a failed export |
| EVIDENCE   | Real export run 20 times: 0 failures   |
| RISK       | Export path only • revert: one commit  |
| UNVERIFIED | Safari's download dialog               |
| CONFIDENCE | high, limited by the evidence          |
|            | raise it by: run the export in Safari  |

RECOMMENDATION

Grade by the weakest area, and always name the one thing that would raise it.

# Keep pending work in the repository, one file per item.

## IN CHAT

Gone when the session ends. When we finally checked, nearly half of our chat-only to-dos were already done or duplicated.



## IN THE REPOSITORY

One small file per item, with a priority and a "done when." Any session can pick it up cold.

PRACTICE 05 • CAST THE CREW

# Orchestration

# Give each agent one job and a name, and pick it by its job.

---

**Scout:** finds where things live and how they work.

---

**Fact checker:** confirms or refutes each claim against the source.

---

**Build fixer:** finds why a check failed, and fixes the cause.

---

**Red team:** breaks the change before users do.

---

**Skeptic:** asks whether we're solving the right problem.

---

**Prose checker:** catches unclear or machine-sounding writing.

# One trio finds your question. The other checks your answer.

THE ADDITIVE TRIO

Seam census, blast radius and contradictions find claims you don't have yet. Run it first.



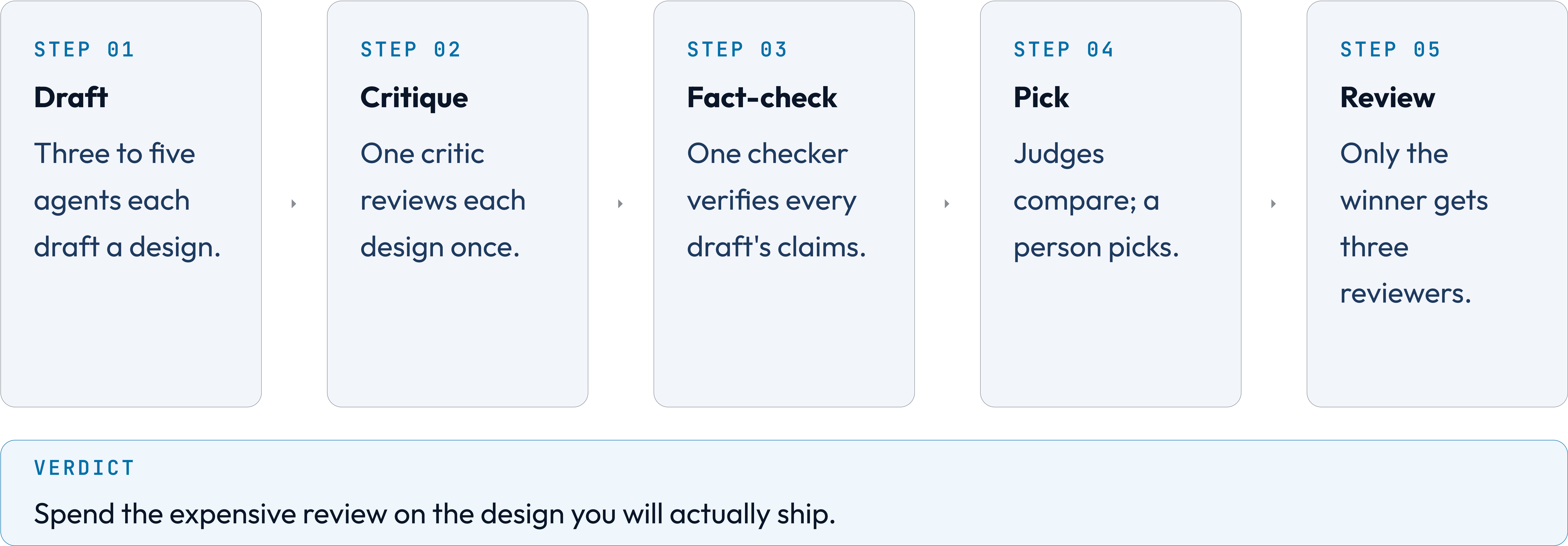
THE ADVERSARIAL TRIO

Red team, inversion and checker attack a claim you already have. Run it on what ships.

WHY IT MATTERS

Checking a wrong answer harder only gives you a better-defended wrong answer.

# Compare several drafts, then review only the winner.



# Five takes can cost more than two good ones.

PRICE PER TAKE

What the pricing page shows: the cost of one request. On simple work, it's the right number to watch.



COST OF THE FINISHED SCENE

The first try, plus every fix, plus your time giving notes. On hard work, this is the number to watch.

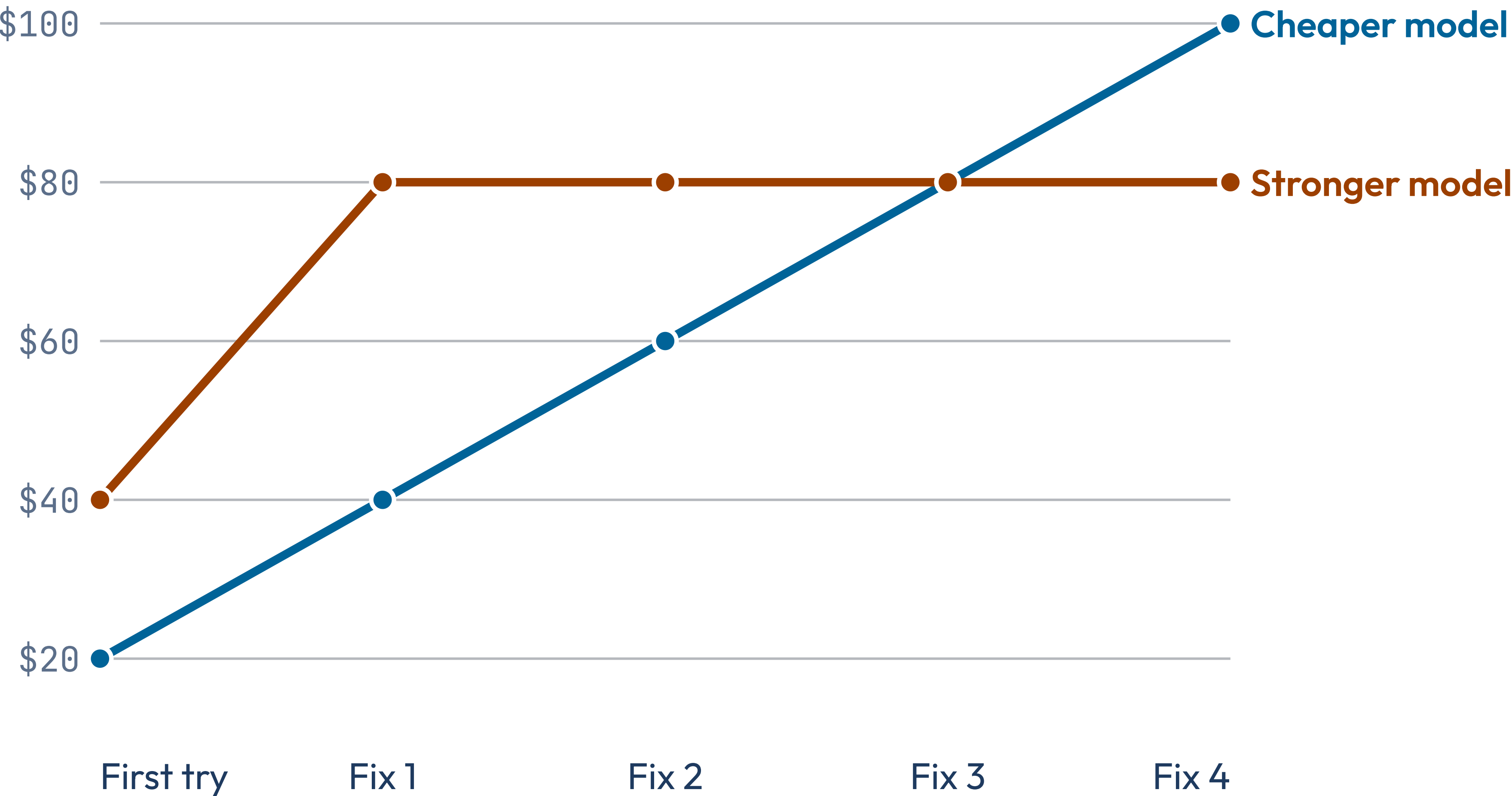
TAKEAWAY

On hard work, count every take before you compare prices.

ORCHESTRATION • CAST THE MODEL

# By the fourth fix, the cheaper model costs more.

*Money spent on 100 hard tasks, round by round*



*Illustrative: each round costs \$20 on the cheaper model and \$40 on the stronger one.*

# Treat agent count like money: estimate it, cap it, and stop early.

---

- 01 Estimate how many agents and roughly what it will cost before you start.

---
- 02 Count across the whole session; small runs add up.

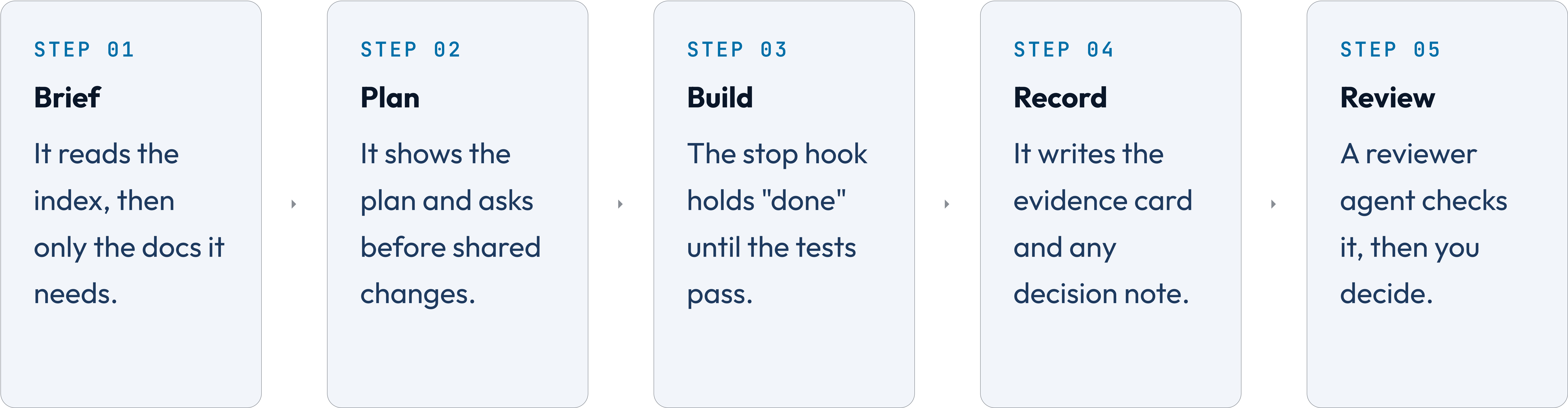
---
- 03 Past about ten agents, a person approves.

---
- 04 Stop when a round changes nothing, usually by round three.

---
- 05 Record what it cost, so you learn what it was worth.

LIVE DEMO · ONE TASK, START TO FINISH

# One task, all five practices, start to finish.



**BOTTOM LINE**  
If it stumbles, the evidence card will say so.

THAT'S A WRAP • WHAT TO DO NEXT

# Make it stick

# Start with one habit per practice, then add a layer each month.

| PRACTICE      | WEEK 1                                      | MONTH 1                                         | QUARTER 1                                      |
|---------------|---------------------------------------------|-------------------------------------------------|------------------------------------------------|
| Context       | <input type="radio"/> A one-page index file | <input type="radio"/> Linked detail docs        | <input type="radio"/> Measure what loads       |
| Autonomy      | <input type="radio"/> Allow and deny lists  | <input type="radio"/> A written stop list       | <input type="radio"/> Approval as a setting    |
| Verification  | <input type="radio"/> Tests on every change | <input type="radio"/> A check that plants a bug | <input type="radio"/> Review risky work twice  |
| Learning      | <input type="radio"/> A decision log        | <input type="radio"/> Pending work as files     | <input type="radio"/> Retest your oldest rules |
| Orchestration | <input type="radio"/> One reviewer agent    | <input type="radio"/> A named roster            | <input type="radio"/> A budget per session     |

☐ Next step

# The practices carry over; "the real thing" changes by field.

| WORK             | THE REAL THING TO CHECK             | A CHECK THAT PROVES IT CAN FAIL          |
|------------------|-------------------------------------|------------------------------------------|
| Data science     | A fixed, versioned evaluation set   | Plant an input that leaks the answer     |
| Data engineering | The target warehouse                | Insert a known-bad row                   |
| Analytics and BI | The published dashboard             | Plant a mismatch with the source         |
| Services         | Staging, or a slice of live traffic | Break a contract on purpose              |
| Tools and CLIs   | The built program's output          | A saved expected output that must change |

# Five guardrails an agent can't forget.

**Constraints:** a gate checks the rule on every build, so nobody has to remember it.

**Budgets:** a ceiling on size, speed and agent count, because agents rarely remove.

**Standards:** a contract per component, so "done" means the same thing every time.

**Visibility:** benchmarks and image diffs turn "feels slower" into a number.

**Boundaries:** a library imports only itself, so a change stays where you made it.

## WHY IT MATTERS

An agent forgets a rule. A gate doesn't.

# Treat shared code like a published library.

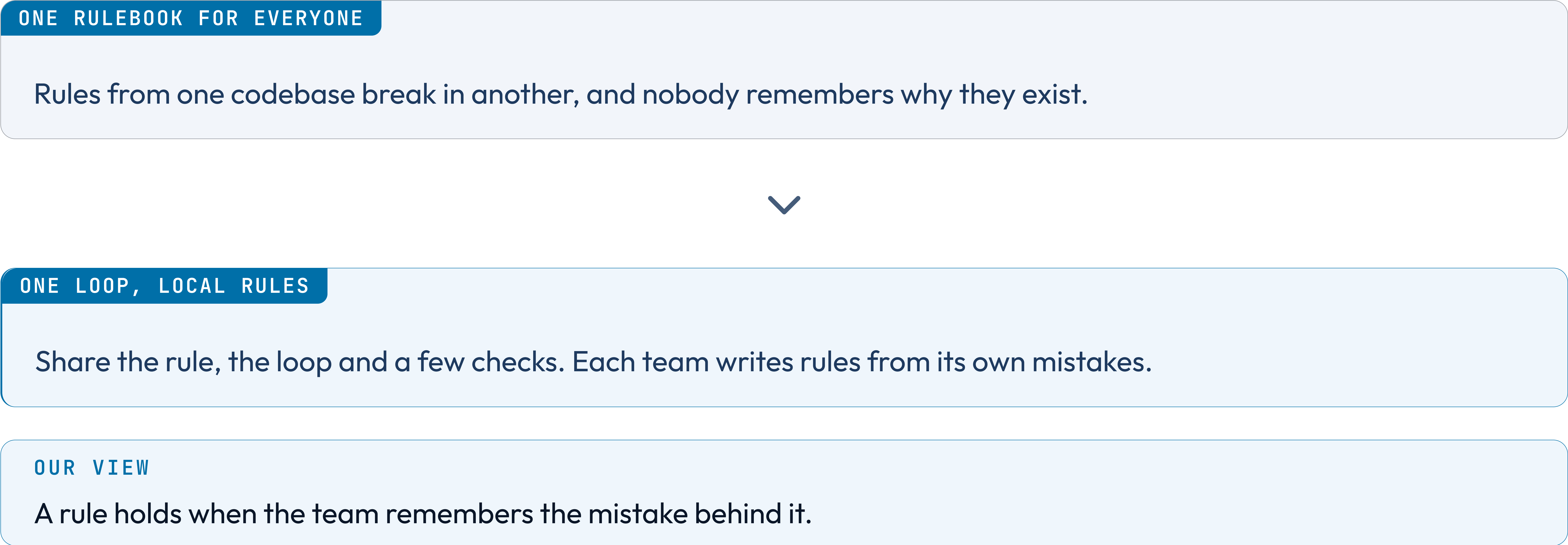
---

- 01 A small public API, and a gate that stops anyone reaching past it.
- 02 Deprecate, don't delete: alias the old name, and fail any build still using it.
- 03 A written contract with conformance tests, so every consumer knows what holds.
- 04 A version and a changelog entry for every change.

## OUR VIEW

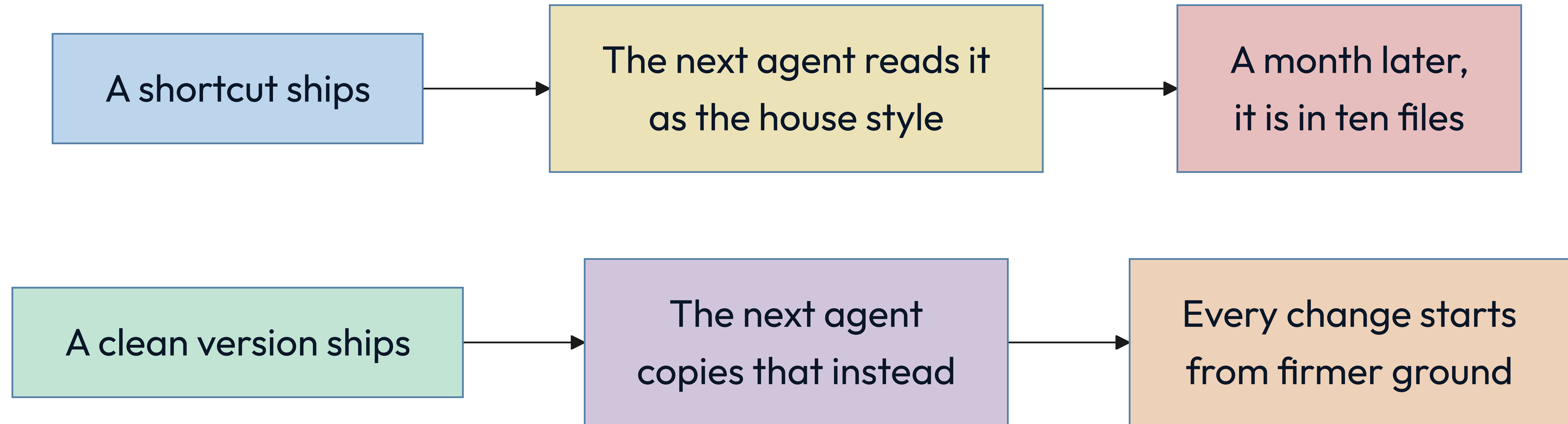
Change a shared library on purpose, as a feature, never as a side effect.

# Share the rule and the loop, and let each team write its own rules.



THE ONE RULE • WHY IT COMPOUNDS

# Agents copy what they find, so quality and jank compound.

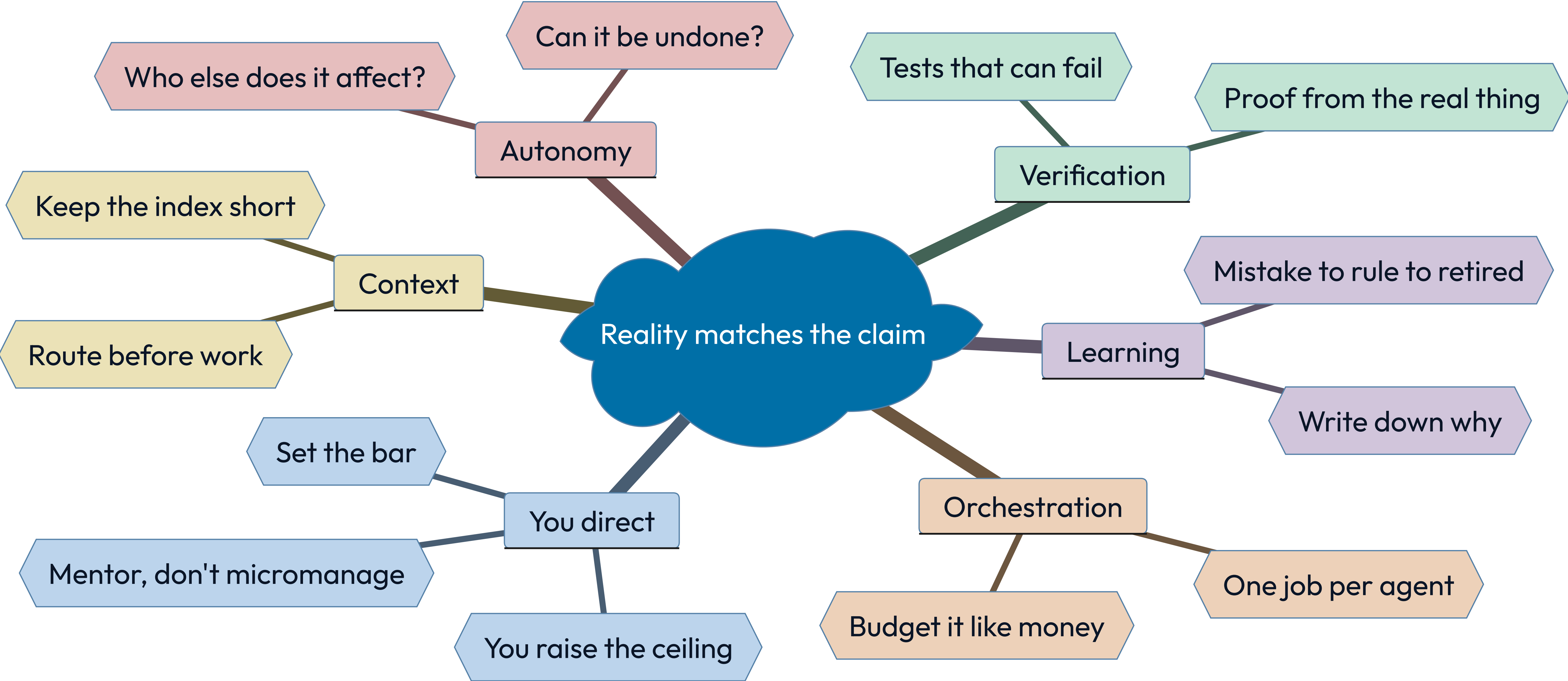


WHY IT MATTERS

Every shortcut you ship becomes context for the next agent.

THE BIG PICTURE · ONE RULE, FIVE PRACTICES

You stop typing code and start directing a system.



YOUR NEXT STEP

# Three moves you can make this week.

---

**01** Rewrite your instruction file as an index, one line per rule.

---

**02** Add one check, then break your code on purpose and watch it catch the bug.

---

**03** Start a decision log: one dated note each time something goes wrong.

THE ASK

Pick one move and try it before next Friday.

CREDITS

Created, produced and directed by Sharmarke Aden.

*Script drafted with Claude Opus 5.5, by Anthropic*



QUESTIONS

# Make reality match the claim

*Quality compounds. So does jank.*

THE STARTER KIT

Copy these, then make them yours

# Seven terms from this talk, in plain words.

C – T

| TERM              | DEFINITION                                              |
|-------------------|---------------------------------------------------------|
| CI, the build     | Automated checks that run on every proposed change.     |
| Context window    | Everything the model can see at one moment.             |
| Hook              | A script the agent tool runs at a fixed moment.         |
| Pull request      | A proposed change, waiting for review before it merges. |
| Session           | One conversation with an agent, from start to finish.   |
| Tautological test | A test that can only agree with the code it checks.     |
| Token             | A chunk of text, about three quarters of a word.        |

# Seven files put all five practices into a repository in five minutes.

| FILE                          | WHERE IT GOES               | PRACTICE                    |
|-------------------------------|-----------------------------|-----------------------------|
| Instruction file              | CLAUDE.md                   | Context, autonomy           |
| Settings                      | .claude/settings.json       | Autonomy, verification      |
| Stop hook                     | .claude/hooks/              | Verification                |
| Reviewer agent                | .claude/agents/checker.md   | Verification, orchestration |
| Evidence card                 | PR template                 | Learning                    |
| Decision note, follow-up file | docs/decisions/, followups/ | Learning                    |

# The instruction file states the rules once and links to the detail.

## *## Always*

- Run the tests before you say anything is done.
- Failing test first, then the fix, then the same test passing.
- "Verified" says where it ran and shows proof from there.  
Otherwise, write UNVERIFIED.

## *## Ask first, even when a rule points at it*

- Shared state: labels, boards, settings others read.
- The CI pipeline or git hooks.
- A number a person set: "about 12" is a decision.
- The meaning of a core doc, including this file.
- Anything irreversible or public.

# The settings file turns the reach test into allow, ask and deny lists.

```
{
  "permissions": {
    "allow": ["Bash(npm test*)", "Bash(git diff*)"],
    "ask":   ["Bash(git push*)"],
    "deny":  ["Bash(git push --force*)", "Read(./env)"]
  },
  "hooks": {
    "Stop": [{ "hooks": [{ "type": "command",
      "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/tests-must-pass.sh"
    }] }]
  }
}
```

# The stop hook keeps the agent working until the tests pass.

```
#!/usr/bin/env bash
# Exit code 2 sends stderr back to Claude and keeps it working.
input=$(cat)
# Already sent back once by this hook? Let it stop.
echo "$input" | grep -q '"stop_hook_active": *true' && exit 0

log=$(mktemp)
if ! npm test --silent >"$log" 2>&1; then
    echo "Tests are failing. Fix them before you finish:" >&2
    tail -20 "$log" >&2
    exit 2
fi
```

# The reviewer agent re-derives every claim and never edits.

```
---  
name: checker  
description: Independent reviewer with fresh eyes. Re-derives  
  every claim and "verified" in a change. Never edits.  
tools: Read, Grep, Glob, Bash  
---  
You are a skeptical reviewer who did not write this change.  
For each claim: find the evidence yourself, run what can be  
run, and check that a test fails when the fix is removed.  
Mark each CONFIRMED, REFUTED or UNVERIFIABLE, with proof.  
Treat "I believe it works" and "CI is green" as unverified.
```

# The evidence card puts the facts in front of every merge decision.

```
Pre-merge: <PR title>
WHAT      <one line: what actually lands>
WHY       <one line: the problem it solves>
EVIDENCE  <what was run or measured, and where>
RISK      <what breaks if wrong> • revert: <how>
UNVERIFIED <caveats that bear on this decision, or none>
CONFIDENCE <low | medium | high | very high>: <weakest area>
          raise it by: <the one thing that would raise it>
```

Areas: evidence • reach • undo • unknowns • review

# The decision note records why, and when to check it again.

```
---  
status: proposed      # proposed | shipped | superseded  
type: decision        # proposal | decision | spec | scoping  
summary: One line a reader can scan in the index  
---  
# <The decision, as a sentence>  
  
Symptom.      What prompted this, with a link or number.  
Cause.        Why. Measured, observed or argued?  
Decision.     What we will do, and what we rejected.  
Enforced by.  The check that holds it, or "discipline only".  
Retest when.  When we check that this still holds.
```

# Follow-up files keep pending work where the next session looks.

```
---  
origin: 1234  
priority: P1  
recorded: 2026-09-25  
---  
# Export drops the last row when the file ends without a newline  
  
why now    – every weekly report is one row short  
where      – src/export/csv.ts  
done when  – a file with no trailing newline exports every row  
verify     – the new unit test, then one real export
```